TRAVAUX PRATIQUES

Introduction aux Conteneurs

& Plateformes Open Source

Master Systèmes Distribués & Télécommunications

Durée : 2 heures | Niveau : Master 1 / Master 2

Prérequis : Linux (notions de base), réseaux TCP/IP

- Comprendre les concepts fondamentaux de la virtualisation par conteneurs
- Distinguer hyperviseurs (Type 1 / Type 2) et conteneurs à partir de schémas d'architecture réels
- Identifier les composants d'une plateforme de conteneurs (moteur, registre, orchestrateur…)
- Déployer et gérer des conteneurs avec Docker
- Comparer les plateformes open source : LXC/LXD, Docker, Podman, Balena
- Comprendre les mécanismes d'isolation du noyau Linux : namespaces, cgroups, SELinux

Partie 1 — Concepts Fondamentaux de la Virtualisation

1.1 Virtualisation traditionnelle vs Conteneurs

La virtualisation système existe depuis les débuts de l'informatique sur mainframe. Trois grandes familles coexistent aujourd'hui, dont les architectures sont illustrées ci-dessous.

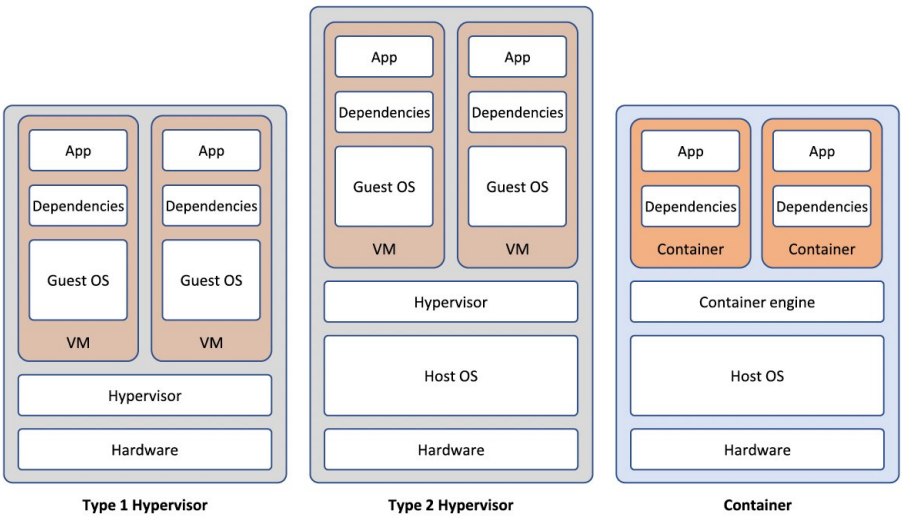

Figure 1 : Comparaison des architectures : Hyperviseur Type 1 / Type 2 / Conteneurs

Hyperviseur de Type 1 (bare-metal)

L'hyperviseur s'exécute directement sur le matériel physique. Sur la Figure 1, chaque VM contient son propre Guest OS, ses dépendances et son application, empilés directement au-dessus de l'hyperviseur et du hardware.

Exemples : VMware ESXi, Microsoft Hyper-V, Proxmox VE, KVM.

Hyperviseur de Type 2 (hosted)

L'hyperviseur est installé par-dessus un OS hôte existant, la Figure 1 montre clairement la couche Host OS supplémentaire entre le hardware et l'hyperviseur, induisant un overhead plus important.

Exemples : VirtualBox, VMware Workstation, QEMU.

Virtualisation par conteneurs

La colonne droite de la Figure 1 montre qu'il n'y a plus de Guest OS par instance. Le Container Engine s'appuie directement sur le Host OS partagé. Chaque conteneur n'embarque que l'application et ses dépendances, expliquant leur légèreté et rapidité de démarrage.

Critère	VM Type 1 / Type 2	Conteneur
Isolation	Complète — OS dédié par VM	Partielle — noyau partagé
Démarrage	Plusieurs minutes	Secondes voire millisecondes
Empreinte mémoire	Élevée (plusieurs Go par VM)	Légère (quelques Mo)
Portabilité	Limitée (dépend du hyperviseur)	Très élevée (image OCI standard)

Surcharge (overhead)	Importante	Quasi nulle
Cas d'usage	Multi-OS, systèmes legacy	Micro-services, DevOps, IoT, edge

Question 1:
En vous appuyant sur la Figure 1 :
a) Expliquez pourquoi l'hyperviseur de Type 2 possède une couche supplémentaire par rapport au Type 1.
b) Pourquoi les conteneurs n'ont-ils pas de Guest OS dans leur pile ?
c) Dans quels cas d'usage préférerait-on un hyperviseur de Type 1 plutôt qu'un conteneur ? (2 critères min.)

1.2 Anatomie d'une plateforme de conteneurs

La Figure 2 présente l'architecture générique d'une plateforme de conteneurs avec l'ensemble des composants et leurs interactions.

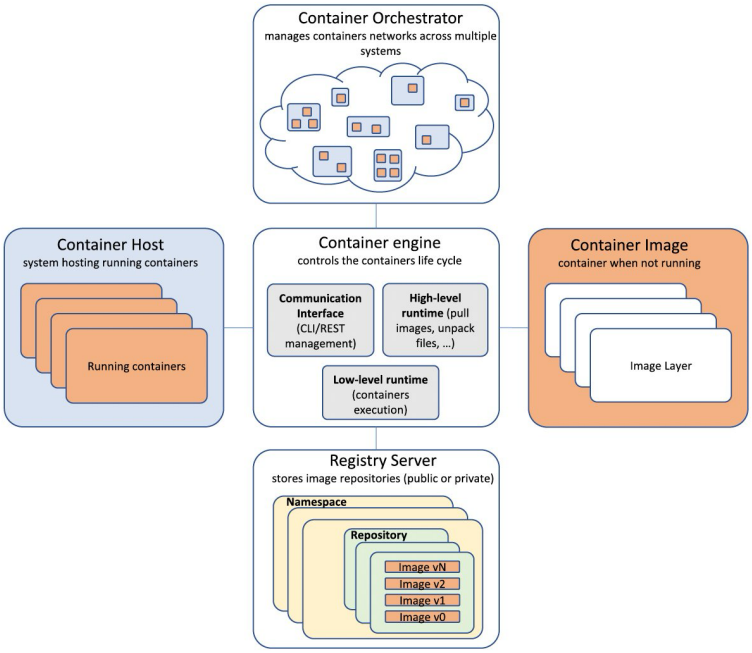

Figure 2 : Architecture générique d'une plateforme de conteneurs

Analyse des composants visibles sur la Figure 2 :

- Container Orchestrator (haut) : gère les réseaux de conteneurs sur plusieurs systèmes. Niveau le plus haut de l'architecture, supervisant la flotte entière.
- Container Host (gauche) : système hébergeant les conteneurs en exécution, empilés et partageant les ressources de la machine.
- Container Engine (centre) : élément central composé de trois sous-parties, interface CLI/REST, runtime haut niveau (pull/push images, réseau, stockage) et runtime bas niveau (exécution effective).

- Container Image (droite) : représentation du conteneur à l'arrêt, organisée en couches (Image Layers) superposées et immuables.
- Registry Server (bas) : stocke les images par namespaces, dépôts et versions taguées (v0, v1, v2, vN).

Question 2: En vous appuyant sur la Figure 2 :

a) Quel composant est responsable du téléchargement (pull) de l'image depuis le Registry Server ?

b) Quelle différence y a-t-il entre le runtime haut niveau et le runtime bas niveau dans le Container Engine ?

c) Pourquoi l'image est-elle représentée comme un empilement de couches (layers) ?

1.3 Mécanismes d'isolation du noyau Linux

La Figure 3 présente l'architecture interne complète d'exécution Docker, détaillant les interactions entre l'espace utilisateur, le moteur Docker et les structures du noyau Linux.

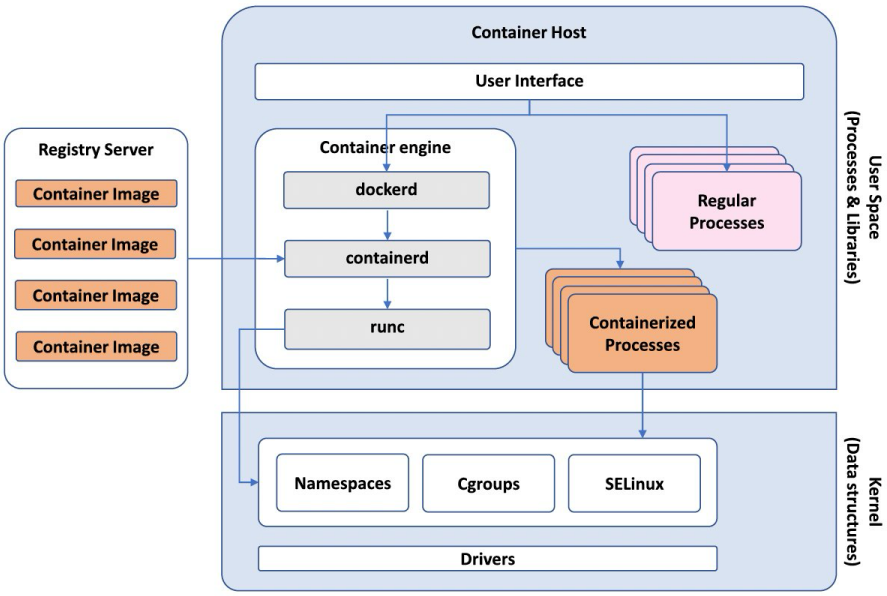

Figure 3: Architecture interne d'exécution Docker : User Space, Container Engine et Kernel Linux

Lecture de la Figure 3 :

- User Space : deux types de processus coexistent : Regular Processes (rose) s'exécutent normalement ; Containerized Processes (orange) sont isolés via le moteur Docker et le noyau.
- Container Engine : dockerd (interface CLI/API REST) → containerd (runtime haut niveau) → runc (runtime bas niveau, interaction directe avec le noyau).
- Registry Server (gauche) : fournit les images à containerd via pull/push.
- Kernel : trois mécanismes : Namespaces, Cgroups, SELinux assurent l'isolation des Containerized Processes.

Mécanisme	Rôle	Exemple concret
Namespaces	Isolation de la visibilité des ressources	PID namespace : un conteneur ne voit que ses propres processus
Cgroups	Limitation des ressources consommées	Restreindre un conteneur à 512 Mo RAM et 0,5 CPU
SELinux	Contrôle d'accès par politique MAC	Interdire l'accès à /etc de l'hôte depuis un conteneur

Question 3:
En vous appuyant sur la Figure 3 :

a) Décrivez le rôle respectif de dockerd, containerd et runc.
b) Expliquez la différence entre Regular Processes et Containerized Processes dans le User Space.
c) Quel mécanisme du noyau limite la quantité de RAM qu'un conteneur peut consommer ?
d) Pourquoi les conteneurs offrent-ils une isolation moins forte qu'une VM ?

Partie 2 — Revue des Plateformes Open Source

2.1 LXC / LXD

LXC (Linux Containers), apparu en 2008, est l'une des premières technologies de conteneurisation Linux. Contrairement à Docker qui cible les conteneurs applicatifs, LXC se positionne sur les conteneurs système, simulant un OS complet.

Caractéristique	Valeur
Année	2008
Type	Conteneurs système (simule un OS complet)
Moteur	liblxc (namespaces + cgroups du noyau Linux)
Extension LXD	API REST, snapshots, gestion VMs, stockage avancé
Réseau	Ponts (bridges), VLAN, macvlan, etc.
Cas d'usage	Legacy OS, serveurs Linux indépendants, labs de recherche
Licence	LGPLv2.1+

LXC a été utilisé dans des travaux de recherche pour émuler un ancien système PowerPC, tirant parti de sa capacité à simuler un OS complet sans la surcharge d'un hyperviseur Type 2.

Commandes LXC / LXD essentielles

```
lxc-create -t download -n mon-conteneur    # Créer un conteneur
lxc-start  -n mon-conteneur                # Démarrer
lxc-attach -n mon-conteneur                # Entrer dans le conteneur
lxc-stop   -n mon-conteneur                # Arrêter
lxc-ls     --fancy                         # Lister avec état
lxc launch ubuntu:22.04 mon-instance       # LXD : créer et démarrer
lxc exec   mon-instance -- bash            # LXD : entrer dans l'instance
lxc snapshot mon-instance snap1            # LXD : créer un snapshot
```

2.2 Docker

Lancé en 2013, Docker est la plateforme de conteneurs la plus adoptée au monde. Son architecture interne est celle décrite dans la Figure 3. Dès la version 0.9, il a remplacé LXC par sa propre bibliothèque libcontainer (Go), ouvrant la voie aux standards OCI.

Caractéristique	Valeur
Année	2013
Type	Conteneurs applicatifs (un processus principal par conteneur)
Architecture	dockerd → containerd → runc (libcontainer en Go)

Comportement par défaut	Stateless, éphémère, portable, léger
Écosystème	Docker Hub, Docker Compose, Docker Swarm
Standards	Compatible OCI (Image Spec, Runtime Spec, Distribution Spec)
Licence	Apache 2.0

Commandes Docker essentielles

```
docker pull ubuntu:22.04
docker run -it ubuntu:22.04 bash
docker run -d -p 8080:80 --name web nginx
docker ps / docker ps -a
docker images
docker logs -f web
docker exec -it web bash
docker stop web && docker rm web
docker build -t mon-app:v1 .
docker image history mon-app:v1
```

2.3 Podman

Podman (Pod Manager), développé par Red Hat, est une alternative moderne à Docker. Sa différence architecturale majeure : il est daemonless, chaque commande Podman est un processus indépendant, sans démon central.

Caractéristique	Valeur
Année	2019
Architecture	Daemonless, pas de démon central
Rootless natif	Exécution complète sans privilèges root
Pods natifs	Groupe de conteneurs partageant un namespace réseau (concept Kubernetes)
Compatibilité OCI	Oui, runc ou crun comme runtime bas niveau
Compatibilité Docker	Très élevée, alias docker=podman fonctionne
Licence	Apache 2.0

Critère	Docker	Podman
Architecture	Démon central (dockerd)	Daemonless (fork/exec par commande)
Rootless	Partiel (ajout récent)	Natif et complet dès l'origine
Pods	Via Docker Compose	Natif (podman pod create)
Export Kubernetes	Non natif	podman generate kube → YAML K8s
Compatibilité OCI	Oui	Oui

Commandes Podman essentielles

```
podman pull nginx
podman run -d -p 8080:80 nginx
podman ps
podman pod create --name mypod
podman run --pod mypod nginx
podman generate kube mypod > pod.yaml
podman play kube pod.yaml
```

2.4 Balena

Balena (anciennement Resin.io, 2013) est une plateforme de conteneurs conçue spécifiquement pour les flottes d'appareils IoT, reposant sur le Projet Moby et optimisée pour les environnements edge à ressources limitées.

Composant	Description
balenaEngine	Moteur allégé compatible Docker/OCI, optimisé edge et IoT
balenaOS	OS léger basé sur Yocto Linux, fiabilité long terme sur embarqué
balenaCloud	Infrastructure complète de gestion de flottes IoT avec OTA updates

Optimisations de balenaEngine par rapport à Docker :

- Binaires plus légers (suppression des fonctionnalités inutiles en IoT)
- Fonctionnalités retirées : Docker Swarm, plugins, overlay networking, cloud logging drivers, stores non-boltdb
- Utilisation RAM et stockage plus conservative
- Bande passante réseau optimisée (deltas d'images pour les mises à jour OTA)

Balena a été adopté dans des projets de recherche IoT pour ses mises à jour OTA atomiques et ses conteneurs stateless sur Raspberry Pi, NVIDIA Jetson, BeagleBone, etc.

2.5 Tableau comparatif des plateformes

Critère	LXC / LXD	Docker	Podman	Balena
Année	2008	2013	2019	2013
Type conteneur	Système	Applicatif	Applicatif	Applicatif
Architecture moteur	liblxc + démon	dockerd/containerd/runc	Daemonless	balenaEngine (Moby)
Rootless natif	Non	Partiel	Oui	Non
Compatibilité OCI	Partielle	Oui	Oui	Oui

Orchestration	LXD cluster	Docker Swarm	Pods + Kube YAML	balenaCloud
Cible principale	Serveurs / legacy OS	DevOps / cloud	Sécurité / entreprise	IoT / edge embarqué
Licence	LGPLv2.1+	Apache 2.0	Apache 2.0	Apache 2.0

Question 4:
Pour chaque scénario, indiquez la plateforme recommandée et justifiez :

a) Déploiement d'une architecture micro-services web en production cloud (haute disponibilité)
b) Gestion d'une flotte de 300 capteurs IoT Raspberry Pi avec mises à jour OTA
c) Émulation d'un ancien serveur PowerPC dans un laboratoire de recherche
d) Environnement CI/CD sécurisé sans accès root sur les agents de build

Partie 3 — Travaux Pratiques : Prise en Main de Docker

3.1 Installation et vérification

```
sudo apt-get update && sudo apt-get install -y ca-certificates curl gnupg
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor
-o /etc/apt/keyrings/docker.gpg
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
sudo usermod -aG docker $USER && newgrp docker
docker --version
docker info
```

Question 5:
Après docker info, relevez :
- Version du moteur Docker
- Runtime de conteneurs utilisé (champ Runtimes)
- Pilote de stockage (Storage Driver)
- Nombre de conteneurs actifs / arrêtés
- Répertoire de données Docker (Docker Root Dir)

3.2 Premier conteneur et cycle de vie

```
docker run hello-world
```

Docker a automatiquement réalisé la séquence suivante :
1. Recherche de l'image hello-world en local (introuvable au premier lancement)
2. Téléchargement de l'image depuis Docker Hub (pull automatique)
3. Création d'un conteneur à partir de l'image
4. Exécution du processus défini et affichage de la sortie standard
5. Arrêt du conteneur (processus terminé)

```
docker ps -a                                # Voir le conteneur arrêté
docker start <id>                           # Redémarrer
docker rm <id>                              # Supprimer
```

3.3 Exploration des images et couches OCI

```
docker pull ubuntu:22.04
docker images
docker image history ubuntu:22.04           # Voir les couches (layers)
docker inspect ubuntu:22.04                 # Métadonnées complètes JSON
```

Question 6 :
(a) Combien de couches possède l'image ubuntu:22.04 ?
(b) Quelle est sa taille totale ?

(c) Expliquez pourquoi les images sont construites en couches immuables et quel avantage cela présente pour le stockage et le téléchargement d'images dérivées.

3.4 Création d'un Dockerfile et construction d'image

```
mkdir ~/tp-docker && cd ~/tp-docker
```

Créez le fichier Dockerfile :

```
FROM ubuntu:22.04
LABEL maintainer="votre.email@domaine.com"
RUN apt-get update && apt-get install -y python3 && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY app.py .
EXPOSE 8080
CMD ["python3", "app.py"]
```

Créez le fichier app.py :

```
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'<h1>Hello depuis mon conteneur Docker !</h1>')
    def log_message(self, *args): pass

print('Serveur démarré sur le port 8080')
HTTPServer(('', 8080), Handler).serve_forever()
```

```
docker build -t mon-serveur:v1 .
docker run -d -p 8080:8080 --name web-test mon-serveur:v1
curl http://localhost:8080
docker logs web-test
```

Question 7 :

(a) Combien de couches ont été ajoutées à ubuntu:22.04 lors du build ?

(b) Que se passe-t-il si vous modifiez uniquement app.py et relancez docker build ? Quelles couches sont reconstruites ?

(c) Quelle directive Dockerfile définit la commande exécutée au démarrage ?

3.5 Isolation des namespaces

```
docker run -it --name iso-test ubuntu:22.04 bash
# Dans le conteneur :
ps aux                                    # Processus visibles
cat /proc/self/cgroup                     # Cgroup du processus courant
hostname && ip addr                       # UTS et NET namespace
```

Depuis un second terminal sur la machine hôte :

```
ps aux | grep bash
docker top iso-test
```

Question 8 :
Comparez ps aux dans le conteneur et docker top depuis l'hôte.
(a) Combien de processus sont visibles dans chaque cas ?
(b) Quel mécanisme du noyau explique cette différence ?
(c) Quelle commande hôte vous a permis de « voir à travers » le namespace PID ?

3.6 Gestion des ressources avec Cgroups

```
docker run -d --memory="256m" --cpus="0.5" --name cgroup-test ubuntu:22.04
sleep 3600
docker stats cgroup-test
docker inspect cgroup-test | grep -A 5 Memory
cat /sys/fs/cgroup/memory/docker/$(docker inspect --format '{{.Id}}' cgroup-
test)/memory.limit_in_bytes
```

Question 9 :
(a) La valeur memory.limit_in_bytes correspond-elle à 256 Mo ?
(b) Comment les cgroups protègent-ils les autres conteneurs contre un conteneur qui consommerait anormalement de la RAM ?
(c) Quelle commande Docker permet de modifier les limites d'un conteneur déjà actif ?

3.7 Persistance des données avec les volumes

```
docker volume create mes-donnees
docker run -d -v mes-donnees:/data --name vol-test ubuntu:22.04 bash -c 'echo
"données persistées" > /data/test.txt && sleep 3600'
docker exec vol-test cat /data/test.txt
docker stop vol-test && docker rm vol-test
docker run --rm -v mes-donnees:/data ubuntu:22.04 cat /data/test.txt
```

Question 10 :
(a) Le fichier test.txt est-il toujours présent après suppression du premier conteneur ? Pourquoi ?
(b) Quelle est la différence entre un volume Docker géré et un bind mount ?
(c) Dans quel cas d'usage choisiriez-vous l'un plutôt que l'autre ?

Partie 4 — Standards OCI et Interopérabilité

4.1 Les spécifications OCI

L'Open Container Initiative (OCI), créée en 2015, définit des standards neutres et portables pour garantir l'interopérabilité entre toutes les plateformes de conteneurs. Docker en est l'un des principaux contributeurs.

Spécification OCI	Objectif	Format / Protocole
Image Format Spec	Format de l'image sur disque	Archive TAR multicouche + fichiers JSON de métadonnées
Runtime Spec	Comment exécuter une image OCI	Fichier config.json + bundle rootfs
Distribution Spec	API de distribution des images	API REST pour push/pull entre registres

Interfaces complémentaires dans l'écosystème Kubernetes :

- CRI (Container Runtime Interface) : plugin Kubernetes permettant d'utiliser différents runtimes (containerd, CRI-O) sans modifier le kubelet.
- CNI (Container Network Interface) : standard CNCF pour configurer les interfaces réseau des conteneurs Linux.
- CNM (Container Network Model) : modèle réseau proposé par Docker, concurrent du CNI.

Question 11: En quoi les standards OCI ont-ils favorisé l'interopérabilité entre LXC et Docker ? Est-il possible de créer un conteneur LXC à partir d'une image Docker ? Si oui, décrivez le principe.

4.2 Exercice — Inspection d'une image OCI

```
docker pull alpine
mkdir ~/oci-inspect && cd ~/oci-inspect
docker save alpine -o alpine.tar
tar xf alpine.tar && ls -la
cat manifest.json | python3 -m json.tool
```

Question 12 :

(a) Combien de couches contient l'image Alpine ?

(b) À quoi sert le champ sha256 dans le manifest.json ?

(c) Quelle garantie technique cela apporte-t-il lors du téléchargement depuis un registre distant ?

Fin du TP — Bonne exploration !